

IF projekt

Tine Zaletelj

17. september 2026

Kazalo

1	Teorija	1
1.1	Vhodni podatki	1
1.2	Rekonstrukcija slike	1
1.3	Filter $\mathbf{R}$ in $\tilde{\mathbf{R}}$	1
1.4	Izbira k_0	2
1.5	Računanje v praksi	2
2	Koda	3
2.1	Uporaba knjižnic	3
2.2	Branje datoteke	3
2.3	Filter $\mathbf{R}$	4
2.4	Izračun transformacije z vsoto	4
2.5	Rekonstrukcija s FFT	5
2.6	Izris rezultatov	6
3	Čas	6
4	Rezultati in diskusija	7
5	Zaključek	8

1 Teorija

Kot vhodne podatke dobimo Radonovo projekcijo-sinogram

$$p(\phi, s) = \int \int f(x, y) \delta(x \cos \phi + y \sin \phi - s) dx dy$$

v datoteki tipa `.dat`. V zgornji enačbi $f(x, y)$ predstavlja intenziteto slike v sivih odtenkih. Velja $0 < f < 1$, kjer 0 predstavlja črno barvo, 1 pa belo. Za vse zglede velja, da je definicijsko območje $f(x, y)$ $[-1, 1]^2$, pri čemer velja, da je funkcija neničelna samo znotraj enotske krožnice.

1.1 Vhodni podatki

V datoteki `ime.dat` imamo n_ϕ vrstic in n_s stolpcev. V prvi vrstici imamo tako podane vrednosti $p(0, -1)$, $p(0, -1 + \Delta s)$, ..., $p(0, 1 - \Delta s)$. Pri tem je $\Delta s = 2/n_s$ korak pri vzorčenju v s -smeri. Podobno za korak v ϕ -smeri zapišemo $\Delta \phi = \pi/n_\phi$.

Tako imamo v i -ti vrstici in j -tem stolpcu vrednost $p(i\Delta\phi, -1 + j\Delta s)$.

1.2 Rekonstrukcija slike

Če imamo podan sinogram $p(\phi, s)$, lahko popolno rekonstrukcijo slike $f(x, y)$ dobimo s pomočjo Fourierove transformacije:

$$\begin{aligned} f(x, y) &= \frac{1}{(2\pi)^2} \int_0^\pi d\phi \int_{-\infty}^\infty dk \tilde{R}(k) \tilde{f}(k \cos \phi, k \sin \phi) \exp(ik(x \cos \phi + y \sin \phi)) = \\ &= \frac{1}{2\pi} \int_0^\pi d\phi [p * R](x \cos \phi + y \sin \phi) = \\ &= \frac{1}{2\pi} \int_0^\pi d\phi \int_{s-1}^{s+1} d\tau [p(\phi, s - \tau) R(\tau)] \Big|_{s=x \cos \phi + y \sin \phi} = \\ &= \frac{1}{2\pi} \int_0^\pi d\phi \int_{-1}^1 d\tau [p(\phi, \tau) R(s - \tau)] \Big|_{s=x \cos \phi + y \sin \phi}. \end{aligned}$$

V prvi vrstici smo zapisali Fourierovo transformacijo, kjer $\tilde{R}$ predstavlja filter v k -prostoru in je bolj podrobno opisan v podpoglavju 1.3. V drugi vrstici smo integral prevedli na konvolucijo funkcij p in R , kjer je R filter v s -prostoru in velja $s = x \cos \phi + y \sin \phi$. Ker nas zanimajo le rezultati znotraj enotske krožnice, velja $-1 < s < 1$.

1.3 Filter R in $\tilde{R}$

Najbolj preprosta oblika filtra v k -prostoru je kar $\tilde{R} = |k|$. V praksi znotraj filtra še porežemo visoke frekvence (za $k > k_0$ velja $k = 0$): $\tilde{R}(k) = |k| \theta(k_0 - |k|)$. Z

uporabo Fourierove transformacije lahko izračunamo

$$\begin{aligned}
R_{k_0}(s) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \tilde{R}_{k_0}(k) e^{iks} dk \\
&= \frac{1}{2\pi} \int_{-\infty}^{\infty} |k| \theta(k_0 - |k|) e^{iks} dk \\
&= \frac{1}{2\pi} \int_{-k_0}^{k_0} |k| e^{iks} dk \\
&= \frac{1}{2\pi} \left[\int_{-k_0}^0 (-k) e^{iks} dk + \int_0^{k_0} k e^{iks} dk \right] \\
&= \frac{1}{2\pi} \int_0^{k_0} k (e^{iks} + e^{-iks}) dk \\
&= \frac{1}{\pi} \int_0^{k_0} k \cos(ks) dk \\
&= \frac{1}{\pi} \left[\frac{k \sin(ks)}{s} \Big|_0^{k_0} - \int_0^{k_0} \frac{\sin(ks)}{s} dk \right] \\
&= \frac{1}{\pi} \left[\frac{k_0 \sin(k_0 s)}{s} - \left(-\frac{\cos(ks)}{s^2} \right) \Big|_0^{k_0} \right] \\
&= \frac{1}{\pi} \left[\frac{k_0 \sin(k_0 s)}{s} + \frac{\cos(k_0 s) - 1}{s^2} \right] \\
&= \frac{1}{2\pi} \left[\frac{2k_0}{s} \sin(k_0 s) - \frac{2(1 - \cos(k_0 s))}{s^2} \right] \\
&= \frac{1}{2\pi} \left[\frac{2k_0}{s} \sin(k_0 s) - \frac{4}{s^2} \sin^2(k_0 s/2) \right].
\end{aligned}$$

V nalogi sem kot dodatek uporabil še Hanningov filter(okno), ki osnovni filter pomnoži še s kosinusom. Uporabil sem še premik, ki gladko zaduši frekvence na robu k_0 :

$$\tilde{R}(k) = |k| \theta(k_0 - |k|) [1/2 + 1/2 \cos(k\pi/k_0)].$$

Opomba: ker je računanje brez FFT porabilo veliko časa, sem uporabo filtra implementiral le za hitro Fourierovo transformacijo.

1.4 Izbira k_0

Ker imamo vzorčenje v smeri s v razdalji Δs , je največja frekvenca, ki jo lahko zaznamo ravno polovična frekvenca vzorčenja, $k_{max} = \pi/\Delta s = n_s \pi/2$, ki jo imenujemo Nyquistova frekvenca. V nalogi sem testiral $k_0 = k_{max}$, $k_{max}/2$, $k_{max}/3$.

1.5 Računanje v praksi

V praksi imamo za izračun integrala v poglavju 1.1 dve možnosti:

- aproksimacija integrala z vsoto,
- FFT(Fast Fourier Transform).

Uvedemo oznaki $p_{ij} = p(i\Delta\phi, -1 + j\Delta s)$, ter $R_{ji}(x, y) = R(s_i + 1 - j\Delta s)$. Za aproksimacijo integrala z vsoto lahko zapišemo

$$f(x, y) = \frac{1}{n_s n_\phi} \sum_{i=0}^{n_\phi-1} \sum_{j=0}^{n_s-1} p_{ij} R_{ji}(x, y) = \frac{1}{n_s n_\phi} \text{Tr}(pR(x, y)).$$

Nato moramo rezultat deliti z maksimumom, da dobimo vrednosti med 0 in 1, $f_{max} = \max_{i,j}(f(x, y)) \rightarrow f_{ij} = f_{ij}/f_{max}$. Nazadnje vse negativne člene ter člene, ki ležijo zunaj enotske krožnice, postavimo na 0. Ker moramo za vsak x in y izračunati sled produkta dveh matrik, je ta način zelo časovno potraten. Postopek si lahko močno pohitrimo, če za vsak (x, y) naredimo FFT.

2 Koda

V programskem jeziku python sem sprogramiral vse, kar je opisano v poglavju 1. Spodaj so prikazani deli kode.

2.1 Uporaba knjižnic

Uporabil sem knjižnice Numpy, Matplotlib ter time. Numpy sem uporabil za branje datoteke ter hitrejšo delo s seznamami oziroma matrikami. Matplotlib sem uporabil za izris vseh slik, time pa za informativno merjenje časa, ki ga funkcija potrebuje za izvedbo. Več o tem v poglavju 3.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time

```

2.2 Branje datoteke

Datoteko sem prebral z uporabo knjižnice Numpy.

```

1 p = np.loadtxt(filename)
2 n_phi, n_s = p.shape

```

V prvi vrstici preberemo datoteko in vsebino shranimo v matriko p , nato pa iz velikosti določimo n_s in n_ϕ .

2.3 Filter R

Filter sem definiriral kot funkcijo:

```
1 def filter(s, k0):
2     R = np.zeros_like(s, dtype=float)
3     nonzero = np.abs(s) > 0
4     s_nz = s[nonzero]
5     R[nonzero] = (1 / (2 * np.pi)) * ((2 * k0 / s_nz) * np.sin(k0 * s_nz) - (4 / (s_nz**2)) * (np.sin(k0 * s_nz) /
6     R[~nonzero] = (k0**2) / (2 * np.pi)
7     return R
```

Za vsak s sem izračunal vrednost $R_{k_0}(s)$ po formuli iz poglavja 1.3. Pri tem sem upošteval le neničelne elemente, saj velja $R_{k_0}(0) = 0$.

2.4 Izračun transformacije z vsoto

```
1 delta_s = 2 / n_s
2 delta_phi = np.pi / n_phi
3 k_max = (n_s * np.pi) / 2
4 velikost = 256
5
6 x = np.linspace(-1, 1, velikost)
7 y = np.linspace(-1, 1, velikost)
8 X, Y = np.meshgrid(x, y)
9
10 koti = np.arange(n_phi) * delta_phi
11 cos_phi = np.cos(koti)
12 sin_phi = np.sin(koti)
13
14 j = np.arange(n_s)
15 shift_s = 1.0 - j * delta_s # s + 1 - j*delta_s
16
17 f = np.zeros((velikost, velikost))
18
19 for a in range(velikost):
20     for b in range(velikost):
21         xi = X[a, b]
22         yi = Y[a, b]
23         if xi**2 + yi**2 >= 1.0:
24             continue
25         s_i = xi * cos_phi + yi * sin_phi
26         arg_R = s_i[np.newaxis, :] + shift_s[:, np.newaxis]
27         R_mat = filter(arg_R, k0)
28         f[a, b] = np.sum(p.T * R_mat) / (n_s * n_phi)
29
30 f[f < 0] = 0.0
```

```

31 f_max = np.max(f)
32 if f_max > 0:
33     f /= f_max

```

Najprej sem definiriral konstante, potrebne za izračun. Za resolucijo sem si izbral 256 pikslov v vsaki smeri. Nato sem ustvaril 256*256 matriko, ki je sestavljena iz samih ničel, nato pa z dvojno for zanko neničelne elemente popravil. Na koncu sem rezultate še normaliziral in negativne elemente postavil na 0.

2.5 Rekonstrukcija s FFT

```

1  delta_s = 2.0 / n_s
2  delta_phi = np.pi / n_phi
3
4  k = np.fft.fftfreq(n_s, d=delta_s) * 2 * np.pi
5  k_max = np.pi / delta_s
6  abs_k = np.abs(k)
7
8  if tip == 'osnova':
9      H = abs_k
10 elif tip == 'hanning':
11     H = abs_k * (0.5 + 0.5 * np.cos(np.pi * abs_k / k_max))
12 H[abs_k > k_max] = 0.0
13
14 p_filt = np.zeros_like(p)
15 for i in range(n_phi):
16     p_fft = np.fft.fft(p[i, :])
17     p_filt[i, :] = np.real(np.fft.ifft(p_fft * H))
18
19 x = np.linspace(-1, 1, grid_size)
20 y = np.linspace(-1, 1, grid_size)
21 X, Y = np.meshgrid(x, y)
22
23 f = np.zeros((grid_size, grid_size))
24 koti = np.arange(n_phi) * delta_phi
25 s_koord = np.linspace(-1, 1, n_s)
26
27 for i, kot in enumerate(koti):
28     s_proj = X * np.cos(kot) + Y * np.sin(kot)
29     f += np.interp(s_proj, s_koord, p_filt[i, :], left=0, right=0)
30 f *= (delta_phi / np.pi)
31
32 f[X**2 + Y**2 >= 1.0] = 0.0
33 f[f < 0] = 0.0
34 f_max = np.max(f)
35 if f_max > 0:
36     f /= f_max

```

Na začetku ponovno definiramo potrebne konstante. Nato s pomočjo knjižnice numpy transformiramo in filtriramo projekcijo. Na koncu rezultate normaliziramo in negativne elemente postavimo na 0.

2.6 Izris rezultatov

Programa iz poglavij 2.4 in 2.5 sem zapakiral v funkciji. Nato sem za različne vrednosti rekonstruiral slike (za klasično transformacijo sem uporabil vrednosti $k_0 = k_{max}$, $k_{max}/2$, $k_{max}/3$; za izračun s FFT pa klasičen in Hanningov filter). Nazadnje sem s pomočjo knjižnice Matplotlib ustvaril 2*3 mrežo in nanjo narisal 6 slik.

```

1 print("k_max")
2 start=time.perf_counter()
3 slika_k_max = klasicno(p, n_phi, n_s, k_0=k_max, velikost=256)
4 konec=time.perf_counter()
5 print(str(konec-start) + "s\n")
6
7 fig, ax = plt.subplots(3, 2, figsize=(8,10))
8
9 ax[0,0].imshow(p, aspect=2/np.pi, cmap='gray', extent=[-1, 1, np.pi, 0])
10 ax[0,0].set_title("$p(\\phi, s)$")
11 ax[0,0].set_xlabel("$s$")
12 ax[0,0].set_ylabel("$\\phi$ [rad]")
13
14 ax[0,1].imshow(slika_han, cmap='gray', extent=[-1, 1, -1, 1])
15 ax[0,1].set_title("$f(x,y)$ fft Hanning")
16 ax[0,1].set_xlabel("$x$")
17 ax[0,1].set_ylabel("$y$")
18 ax[0,1].set_xlim([-1,1])
19 ax[0,1].set_ylim([-1,1])

```

3 Čas

Preden je celotna koda delovala, sem poganjal vsako funkcijo posebej in opazil sem, da se koda, ko uporabim FFT, izvede precej hitreje kot koda s klasično transformacijo. Zato sem se odločil beležiti čas izvajanja s pomočjo knjižnice time. Rezultati so prikazani v tabeli 3.

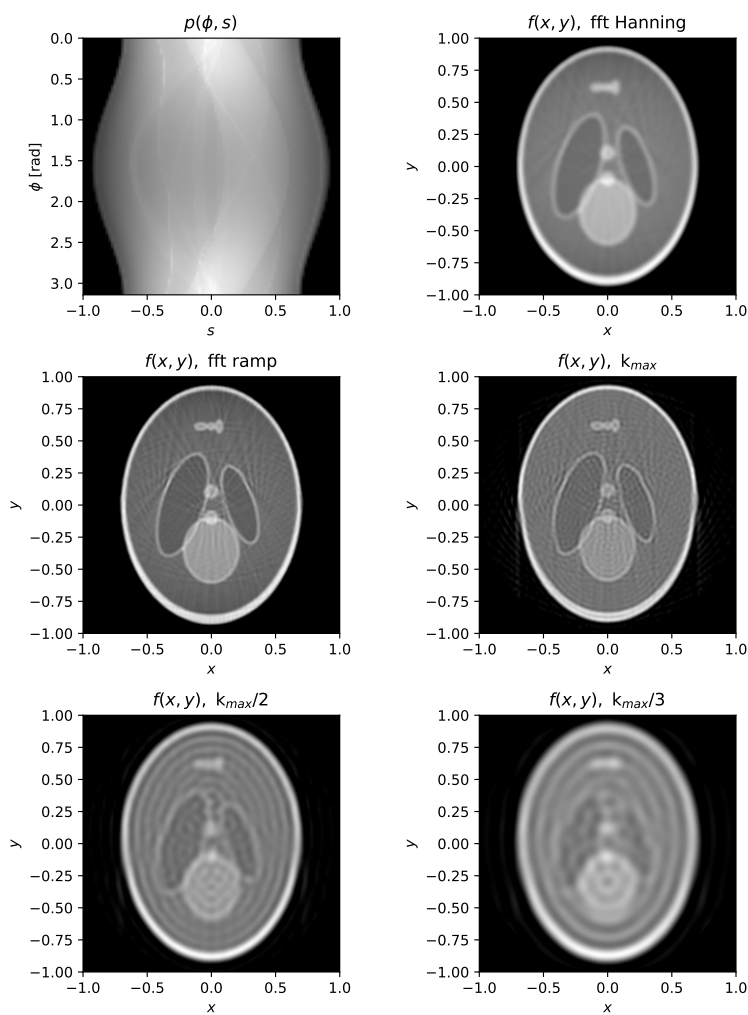
Ime datoteke	FFT[s]	$k_0=k_{max}$ [s]	$k_0=k_{max}/2$ [s]	$k_0=k_{max}/3$ [s]
phantom.dat	0.047	07.67	08.02	08.66
sg2.dat	0.088	30.95	32.36	33.79
sg.dat	0.090	38.25	38.47	38.71

Opazimo, da je hitra Fourierova transformacija vsaj za dva velikostna reda hitrejša od klasične transformacije, rezultati pa so izjemno podobni.

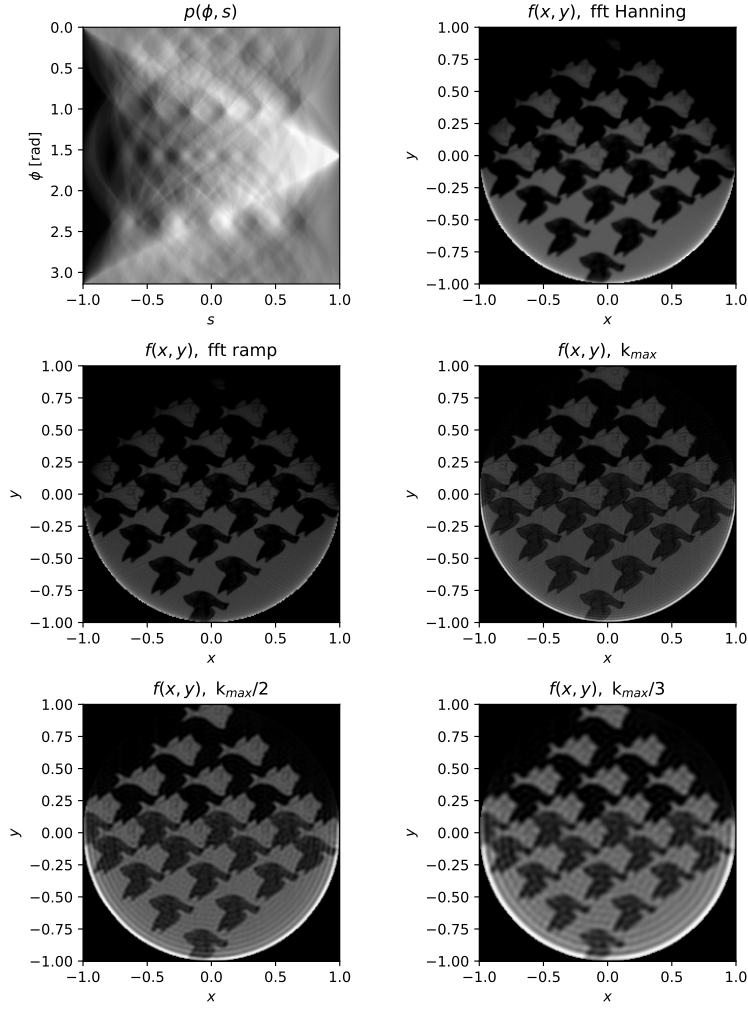
Uporaba FFT s filtrom je porabila približno isto časa kot brez filtra-razlika je bila $\sim 10^{-3}$ s.

4 Rezultati in diskusija

Na sliki 1 je prikazana transformacija datoteke `phantom.dat`. Lahko vidimo, da je kvaliteta nekoliko slabša kot original, predstavljen v navodilih. To je verjetno zaradi izbire resolucije ter aproksimacije integrala z vsoto. Na sliki je prikazana rekonstrukcija datoteke `sg2.dat`, na sliki 3 pa rekonstrukcija datoteke `sg3.dat`.



Slika 1: Rekonstrukcija datoteke `phantom.dat`



Slika 2: Rekonstrukcija datoteke `sg2.dat`

5 Zaključek

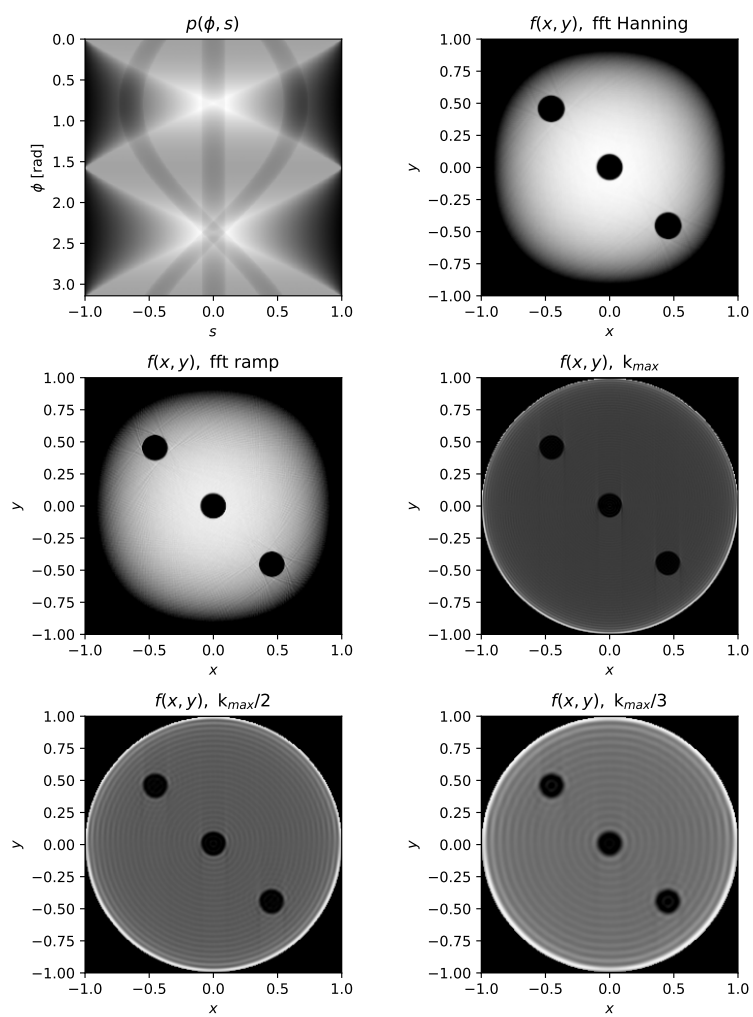
Uspešno smo rekonstruirali 3 Radonove projekcije, čeprev smo zaradi aproksimacije integrala z vsoto izgubili nekaj kvalitete. Uporabili smo dva različna načina, in sicer z vsoto ter s FFT. Pri FFT smo preizkusili dva različna filtra, ki sta nekoliko spremenila rezultat. Hanningov filter je sproduciral nekoliko svetlejši rezultat, poleg tega pa je nekoliko zameglil ostre prehode med svetlimi in temnimi območji.

Pri rekonstrukciji z vsoto smo eksperimentirali z različnimi vrednostmi za k_0 .

Manjši kot je bil k_0 , bolj svetla je bila rekonstrukcija, ampak smo pri rekonstrukciji datoteke `phantom.dat` dobili precej manj ostre robove.

Na koncu smo si pogledali še čas, ki ga je Python porabil za rekonstrukcijo. Metoda s FFT se je v tem primeru izkazala za veliko boljšo, ker je porabila za faktor 10^{-3} manj časa.

Obe metodi sta se izkazali za ustrezni, ampak bi v praksi zaradi hitrosti raje uporabljal metodo FFT. Naše datoteke so bile razmeroma majhne, ampak bi v primeru večje datoteke s hitro Fourierovo transformacijo prihranil ogromno časa.



Slika 3: Rekonstrukcija datoteke sg3.dat